linuxmuster.net 7.4

Linux-Workshop - ZSL ES 19.10.2026

Ubuntu 26.04 · OPNsense 26 · linuxmuster-base & -linbo 7.4

Agenda

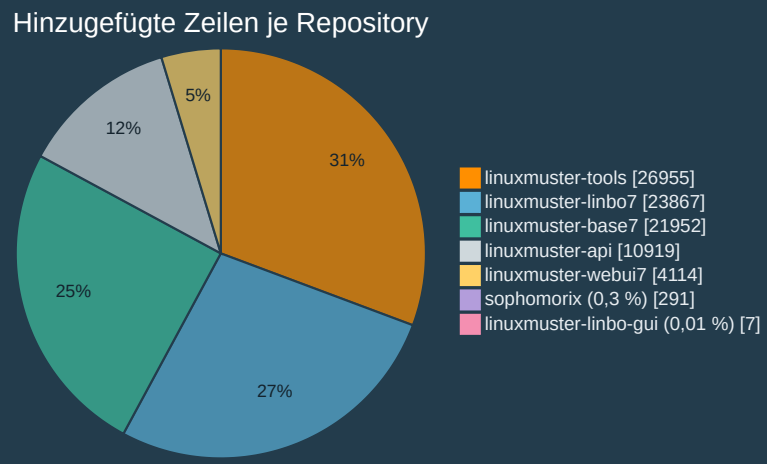
1. Überblick und Zahlen
2. Upgrade auf 7.4
3. Setup und Servermanagement
   - Basis, Firewall-API, Zertifikate, Quota
4. Imaging und Clientmanagement
   - LINBO: Komponenten & Kernel, aria2c, Treiberprofile, linbo-remote
5. Betrieb und Fehlersuche
6. Zusammenfassung, Fragen

1 · Überblick

7.4 in Zahlen Stand 15.10.2026

1.291	121	41	1.204
Commits	Issues	Pull Requests	Dateien geändert

- Netto rund **37.000 neue Zeilen** (+95.027 / −57.781) in elf Repositories
- Größte Anteile: **tools** (27 k), **linbo7** (24 k), **base7** (22 k), **api** (11 k), **webui7** (4 k)
- Ein großes **Wartungs- und Modernisierungsrelease**
- **KI-Unterstützung** (Claude Code)



Vergleich 7.4-Linie gegen 7.3-Linie ab Abzweigpunkt. Zeilen und Dateien ohne generierte Sphinx-Doku (tools, webui7) und shunit2 (linbo7).

Drei Themenbereiche

Upgrade 7.3 → 7.4

`linuxmuster-release-upgrade`

Nicht interaktives Distributionsupgrade und Migration der Konfiguration

Setup und Servermanagement

`linuxmuster-base7`, `-common`, `-prepare`

Python-Neustruktur, Firewall per REST-API, Zertifikate, Quota

Imaging und Clientmanagement

`linuxmuster-linbo7`

LinboFS, Kernel, aria2c, Windows-Treiberprofile, `linbo-remote`

2 · Upgrade 7.3 → 7.4

Voraussetzungen

- **OPNsense ≥ 26**, vorher aktualisieren
- Bestehende Firewallregeln **ins neue Format migriert** (OPNsense 26.1 Rule Migration)
- 7.3-Server **auf aktuellem Stand**, ggf. vorher rebooten
- Internetverbindung des Servers
- Umstieg von 7.3 auf 7.4 geht **nur zusammen mit Ubuntu 26.04**, weil sich die Paketstruktur grundlegend ändert
- Bestehende Konfiguration bleibt erhalten, wird ggf. migriert

> Vor dem Upgrade: **Snapshot** von Server und Firewall.

Upgradeprozess

```
# als root auf der Serverkonsole, bei SSH in tmux
tmux new -s upgrade
linuxmuster-release-upgrade      # Sicherheitsabfrage mit "YES" bestätigen, oder per "--force" überspringen
# ... Kaffee holen ...
reboot
```

- prüft, ob Ubuntu 26.04 als Upgrade-Release verfügbar ist,
- stoppt LINBO-Imaging-Dienste `torrent` und `multicast`,
- prüft, ob Grub-Boot-Laufwerke hinterlegt sind, falls nicht, wird es abgefragt,
- entfernt störende Pakete (`modemmanager`, `cloud-init`),
- startet das Ubuntu-Release-Upgrade noninteraktiv (`DistUpgradeViewNonInteractive`),
- maskiert `isc-dhcp-server6`, deaktiviert `systemd-resolved` Stub-Listener und aktiviert `systemd-networkd-wait-online` Dienste,
- aktualisiert die linuxmuster.net-Pakete explizit, um Abhängigkeiten sauber aufzulösen (z.B. `dracut`),
- stellt Quota auf dem Root-Dateisystem auf das **interne ext4-Feature** um, andere Dateisysteme bleiben bei externen Quota-Dateien (Details: Folien „Quota“)

Nach dem Upgrade prüfen

```
cat /etc/os-release | grep VERSION          # Ubuntu 26.04
dpkg -l | grep linuxmuster                  # 7.4.x
systemctl status isc-dhcp-server samba-ad-dc
linuxmuster-opnsense-apikey --check         # API-Key der Firewall gültig?
ls /var/log/linuxmuster/                    # Logs der Werkzeuge
```

- Migration der Konfiguration, wo nötig (Bestandsserver werden auf denselben Stand gebracht):
  - Fix-Skripte im `postinst` sind **idempotent**:
    - `fix-cupsd.conf_189_190.sh`: Druckerfreigabe `@printing` ergänzt (#189), `/admin/conf` nur noch mit Anmeldung (#190)
    - `fix-systemd-resolved_203.sh`: hebt die Maskierung von `systemd-resolved` und `networkd-wait-online` auf, Stub-Listener per Drop-in aus (#203)
    - `fix-dhcpd-restart-on-failure_210.sh`: `isc-dhcp-server` startet bei Fehlschlag beim Boot neu (#210)
    - `fix-samba-interfaces_212.sh`: Samba registriert nur noch die Serveradresse im DNS (#212)

3 · Setup und Servermanagement

Neue Python-Basis

- Der gesamte Python-Code von **linuxmuster-base7** ist nach **Debian Python Policy** neu strukturiert
- `functions.py` wurde zum Paket `functions/` mit 8 Submodulen:

Modul	Inhalt
`core`, `files`	Logging, setup.ini, Datei-I/O
`network`	Subnetze, Geräte-CSV, Validierung
`samba`, `linbo`, `certs`	AD-Helfer, LINBO/GRUB, Zertifikate
`remote`, `security`	Firewall-API / SSH, Passwörter

- **Verbesserte Sicherheit:** Passwortdateien werden von Anfang an mit restriktiven Rechten angelegt, Shell-Aufrufe laufen ohne Shell (`subprocess.run()` mit Listenargumenten)
- Bestehende Importe bleiben gültig: `from linuxmuster_base7.functions import X`

Firewall: REST-API statt SSH-Kommandos

Neu in 7.4

- OPNsense wird über die **REST-API** angebunden
- Robuster als Kommandos über SSH
- Verständliche Fehlermeldung bei Auth-Fehlern (statt irreführender Versionsmeldung)
- Eingebaute Firewallregeln im **neuen UUID-Schema**: keine Migration nach Neuinstallation nötig
- Squid-Plugins in der Firmware-Liste korrekt registriert

Weiterhin SSH/SCP

Die **Ersteinrichtung** (`m_firewall.py`) bleibt SCP-basiert, da es zu diesem Zeitpunkt noch keinen API-Key gibt, d.h. gepatchte Konfigration wird per SCP hochgeladen.

API basiert gilt daher für den laufenden Betrieb (Subnetze, Reset, Zertifikate), nicht für das Setup.

API-Key prüfen und erneuern

Problem: HTTP 401 „Authentication Failed", z. B. nach Wiederherstellen einer älteren Firewall-Sicherung.

```
linuxmuster-opnsense-apikey              # ohne Option: Hilfe
linuxmuster-opnsense-apikey --check      # nur den aktuellen Key testen
linuxmuster-opnsense-apikey --deploy     # neuen Key ausrollen, falls der aktuelle nicht geht
linuxmuster-opnsense-apikey --deploy --force   # Key in jedem Fall ersetzen
```

- Der Key wird per ssh auf der Firewall angelegt und in `firewall.api.ini` gespeichert
- Ablauf zeigen: Key auf der Firewall deaktivieren → `--check` schlägt fehl → `--deploy` → `--check` ok
- `linuxmuster-import-subnets` meldet Auth-Fehler jetzt klar

Firewall-Reset und Subnetze

```
linuxmuster-opnsense-reset      # zurück in den Zustand nach dem Setup
linuxmuster-import-subnets      # Subnetze: DHCP, Routen, NAT, NTP
```

- `opnsense-reset` spielt die Default-Konfiguration ein, legt den **Kerberos-Keytab** neu an und wartet, bis die Firewall wieder da ist
- Neu: **Subnetze werden nach dem Reset automatisch neu importiert**. Statische Routen und Outbound-NAT für weitere Subnetze gingen vorher verloren.
- Keytab/SSO: Squid wird neu gestartet, *nachdem* der Keytab existiert
- `isValidHostIpv4()` prüft IP-Adressen jetzt **gegen ihr Subnetz** (#205)
  - Vorher: Adressen auf `.0` oder `.255` galten pauschal als Netz- bzw. Broadcastadresse, das stimmt nur bei /24
  - In `10.0.0.0/23` ist `10.0.0.255` eine normale Host-Adresse, wurde aber abgelehnt
  - In `10.0.0.0/26` ist `10.0.0.63` die Broadcastadresse, wurde aber akzeptiert
  - Jetzt: echte Netz- und Broadcastadresse aus der Präfixlänge berechnet, Adressen außerhalb des Subnetzes abgelehnt
  - Gilt für Server-, Firewall-IP und DHCP-Bereich im Setup sowie Router und DHCP-Bereich in `import-subnets`

Zertifikate

- Neu erzeugte Zertifikate enthalten den **Subject Alternative Name**, wie ihn moderne Browser und Betriebssysteme erwarten
- **CA-Erneuerung repariert**: Sie brach mit openssl-Fehler ab bzw. installierte das neue Zertifikat nicht im System-Trust-Store
- CA-Erzeugung in gemeinsame Funktionen konsolidiert
- Dateinamen der Serverzertifikate nach **Servername**

DEMO

```
openssl x509 -in /etc/linuxmuster/ssl/cacert.pem \
  -noout -ext subjectAltName

# CA, Server- und Firewall-Zertifikat
linuxmuster-renew-certs -c all
```

Nie `-c ca` **allein** auf von 7.3 aktualisierten Servern: Der Subject der CA ändert sich, Server- und Firewall-Zertifikat werden ungültig (#216).

Quota: zwei Verfahren in ext4

	extern (bisher überall)	**intern** (neu in 7.4)
Quota-Daten	Dateien `aquota.user`, `aquota.group` im Dateisystem	versteckte Quota-Inodes, Feature `quota` im Superblock
Einschalten	nach jedem Boot `quotaon` (`quotaon@<mount>.service`)	automatisch durch den Kernel beim Einhängen
fstab in 7.3	`usrjquota=aquota.user,grpjquota=aquota.group,jqfmt=vfsv0`	–
fstab in 7.4	`usrquota,grpquota`	`usrquota,grpquota`
Kernel meldet	`Quota mode: writeback`	`Quota mode: journalled`

Haken: Das Feature setzt `tune2fs -O quota`, und das geht nur an einem **nicht eingehängten** Dateisystem. Root ist im laufenden Betrieb immer eingehängt.

Rolle von dracut: Ubuntu 26.04 baut das initramfs mit **dracut** statt initramfs-tools. `linuxmuster-prepare` liefert das Modul `99linuxmuster`: Ein pre-mount-Hook ruft `tune2fs -O quota` vor dem Einhängen auf. dracut ist nur der **Weg**, nicht der Grund der Umstellung.

Quota-Migration 7.3 → 7.4 mit LVM

1. `linuxmuster-release-upgrade`: Ubuntu 26.04, dann linuxmuster-Pakete 7.4: `linuxmuster-base7` bringt **dracut**, `linuxmuster-prepare` das Modul `99linuxmuster`
2. `e_fstab.py` (vom Upgrade aufgerufen):
   - Root ohne Feature: initramfs neu bauen (`dracut --add linuxmuster`)
   - fstab: `usrjquota=`, `grpjquota=`, `jqfmt=` raus, `acl,usrquota,grpquota` rein
   - LVM-Volumes: bleiben extern, Quota neu aktiviert (`quotacheck`, `quotaon`), sofort oder nach dem Reboot
   - `quotaon-root` / `quotaon.service` maskiert, `quotaon@<mount>` nur bei Volumes mit Feature
3. **Reboot**: Der Hook setzt das Feature auf Root. Die Volumes in `vg_srv` sind im initramfs nicht aktiviert und damit unsichtbar.

Ergebnis

Mount	Verfahren
`/` (`vda2`)	intern
`/var`, `/srv/linbo`, `/srv/samba/global`, `/srv/samba/schools/default-school` (`vg_srv`)	extern, `quotaon@….service` aktiv

Quota ist überall aktiv, `systemctl --failed` ist leer.

```
tune2fs -l /dev/mapper/vg_srv-global | grep features
quotaon -pu /srv/samba/global
systemctl list-units 'quotaon@*'
journalctl -b -k | grep 'Quota mode'
```

Weitere Verbesserungen im Betrieb

- NTP (ntpsec): veraltete `notrap` / `nopeer` entfernt, saubere Erststarts ohne Fehler in `/var/log/ntpsec`
- **systemd-resolved** auf Samba-DCs: Reparatur für durch ältere Upgrades maskierte Units
- **SSH**: `dsa` aus den LINBO-Hostkey-Typen entfernt
- **cups**: Anpassung an aktuelle Version (#189)
- Firewall-Einrichtung: **NoProxy**-Alias korrigiert, Plugins `os-squid`, `os-web-proxy-sso`, `os-freeradius` werden korrekt registriert

4 · Imaging und Clientmanagement

LINBO: Basis überarbeitet

- Bauprozess nutzt jetzt komplett **reguläre Ubuntu-Pakete und -Komponenten**
- **Standard-Ubuntu-Kernel** statt selbst kompiliertem
- Torrent-Verteilung mit **aria2c** statt `ctorrent` (Details: nächste Folien)
- Hardwareerkennung über **systemd-udevd**
- **Erweiterbar:** Apps des Servers lassen sich per `harvest-app.sh` „ernten“ und über `update-linbofs`-Hooks ins `linbofs` einbauen (Demo folgt)
- Versionsschema: **4.3.x → 7.4.x**

Warum?

Wartung und Absicherung werden deutlich einfacher, denn Kernel-Updates, Firmware und Module kommen aus der Distribution.

Der vorkompilierte Kernel-Pfad (4.3) wird nicht auf 7.4 portiert.

aria2c: globale Optionen

`/etc/default/linbo-torrent`

Option	Bedeutung
`ARIA2C_GLOBAL_OPTS`	*immer, beim Laden und Seeden*
`--enable-color=false`	keine Farbcodes in Konsole und Log
`--enable-dht=false`	kein DHT[1], Peers nur über Tracker und LPD[2]
`--disable-ipv6=true`	nur IPv4
`--bt-enable-lpd=true`	**neu:** Peers im selben Segment per Multicast finden (LPD[2])
`--bt-tracker-interval=300`	**neu:** alle 5 statt 30 Minuten beim Tracker melden

[1] DHT: *Distributed Hash Table*, trackerlose Peer-Suche übers Internet

[2] LPD: *Local Peer Discovery*, Peer-Suche per Multicast im lokalen Netzsegment

linuxmuster.net

aria2c: Download- und Seed-Optionen

Option	Bedeutung
`ARIA2C_DWNLD_OPTS`	*Client lädt ein Image*
`-c`	abgebrochenen Download fortsetzen
`--console-log-level=notice`	nur Meldungen ab „notice“ ausgeben
`--show-console-readout=true` `--summary-interval=3`	Fortschrittszeile, alle 3 Sekunden eine Zusammenfassung
`--seed-time=0`	nach dem Download beenden, danach seedet `linbo_seeder`
`ARIA2C_SEED_OPTS`	*Server und Clients seeden*
`-V`	vorhandene Datei gegen den Torrent prüfen, dann seeden ohne Download
`--seed-ratio=0.0`	unbegrenzt seeden, unabhängig vom Verhältnis Upload zu Download

Wer nutzt was?

Rolle	Optionen	gestartet von
Server seedet	GLOBAL + SEED	`linbo-torrent` (tmux, als `nobody`)
Client lädt	GLOBAL + DWNLD	`linbo_download_image`
Client seedet	GLOBAL + SEED	`linbo_seeder` (tmux)

DEMO · aria2c-Seeder beobachten

Server

```
# linbo-torrent status
debian_qcow2_torrent: 1 windows (created Sun Oct 18 12:46:3
debian_qdiff_torrent: 1 windows (created Sun Oct 18 12:46:3
win11_qcow2_torrent: 1 windows (created Sun Oct 18 12:46:33
 ...

# ps -o user,args -C aria2c
nobody  /usr/bin/aria2c --enable-color=false --enable-dht=f
        --disable-ipv6=true --bt-enable-lpd=true
        --bt-tracker-interval=300 -V --seed-ratio=0.0
        debian.qcow2.torrent

# linbo-torrent attach debian.qcow2
[NOTICE] Überprüfung erfolgreich abgeschlossen.
         Datei=/srv/linbo/images/debian/debian.qcow2
[NOTICE] IPv4 BitTorrent: Lauschen am TCP-Port 6960
[#0b1374 SEED(1.0) CN:0 SD:0 UL:119MiB(2.1GiB)]
FILE: /srv/linbo/images/debian/debian.qcow2
```

Lösen mit CTRL+B+D

Client

```
# linbo-ssh client1
# tmux ls
debian_qcow2_torrent: 1 windows (created Sun Oct 18 14:16:2
debian_qdiff_torrent: 1 windows (created Sun Oct 18 14:16:4
win11_qcow2_torrent: 1 windows (created Sun Oct 18 14:16:12

# tmux attach -t debian_qcow2_torrent
[NOTICE] Downloading 1 item(s)
[#a61245 0B/2.1GiB(0%) CN:0 SD:0 DL:0B]
  [Checksum:#a61245 1.7GiB/2.1GiB(82%)]
[NOTICE] Überprüfung erfolgreich abgeschlossen.
         Datei=/cache/debian.qcow2
[#a61245 SEED(0.0) CN:0 SD:0]
FILE: /cache/debian.qcow2
```

Lesehilfe: `0B/2.1GiB(0%)` geladen/Gesamtgröße · `DL` Download-Rate · `Checksum` Prüfung der vorhandenen Datei (`-V`) · `SEED(1.0)` Verhältnis Upload zu Dateigröße · `CN` Verbindungen · `SD` Seeder · `UL` Upload-Rate (gesamt)

linuxmuster.net

opentracker, Hashdateien und Torrent-Werkzeuge

Neu: Hashdateien

- Zu jedem Torrent legt `linbo-torrent` eine Datei `<image>.hash` mit dem Info-Hash an, z. B. `ubuntu.qcow2.hash`
- Der Client lädt die Hashdatei vom Server und vergleicht sie mit dem Hash seiner lokalen `.torrent`-Datei
- Weichen sie ab, lädt der Client das **neue Image** herunter

Neu: Whitelist

- `/srv/linbo/images/opentracker/torrent.whitelist` enthält alle Info-Hashes aus den `*.hash`-Dateien
- opentracker bedient **nur diese Torrents** (`access.whitelist` in `opentracker.conf`)
- Aktualisiert bei `linbo-torrent start|restart|create` oder mit `linbo-torrent update-otr`

Neu: `buildtorrent` und `btcheck`

Aufgabe	7.3 (`ctorrent`)	7.4
Torrent erzeugen	`ctorrent -t -u <url> -s <torrent> <image>`	`buildtorrent --announce <url> <image> <torrent>`
Torrent prüfen	`ctorrent -c <torrent>`	`btcheck -v <torrent>`
Info-Hash lesen	–	`btcheck -ni <torrent>`

`<url>` = `http://<serverip>:6969/announce`

Im Alltag reichen die Wrapper:

```
linbo-torrent create ubuntu.qcow2
linbo-torrent check ubuntu.qcow2
```

DEMO · linbofs um eine App erweitern

Beispiel: `/usr/bin/fwupdtool` (Firmware-Werkzeug aus dem Paket `fwupd`) auf die Clients bringen

1 · Installieren und ernten

```
apt-get install fwupd

/usr/share/linuxmuster/linbo/harvest-ap
  /usr/bin/fwupdtool \
  /etc/fwupd /usr/share/fwupd
```

Bibliotheken werden per `ldd` ermittelt, schon vorhandene ausgelassen. Ergebnis: `fwupdtool.tar.xz` in `/var/cache/linuxmuster/linbo/apps/`, weitere Argumente werden mitgepackt.

2 · Hook anlegen

```
#  … /hooks/update-linbofs.pre.d/10_fwu
#!/bin/bash
tar --keep-directory-symlink -xf \
  /var/cache/linuxmuster/linbo/apps/fwu
```

```
chmod +x /var/lib/linuxmuster/hooks/\
  update-linbofs.pre.d/10_fwupdtool
```

Der Hook läuft im Wurzelverzeichnis des `linbofs` und muss **ausführbar** sein.

3 · Einbauen und prüfen

```
update-linbofs
# Ausgabe: "Executing pre hookfile  … "

# Client neu starten, in LINBO
# per Konsole:
fwupdtool get-devices
```

linbo-remote und `--dry-run`

`linbo-remote` ist komplett in **Python** neu geschrieben, gleicher äußerer Aufruf.

```
linbo-remote -i r100-pc01 -c reboot --dry-run     # zeigt nur, was passieren würde
linbo-remote -i r100-pc01 -c reboot               # führt aus
```

- WOL wird über **Samba AD** aufgelöst
- Ein Logfile **pro Lauf**: `<host>_linbo-remote_<YYYYmmddHHMMSS>.log`, alte Logs werden aufgeräumt
- JSON-Laufprotokoll: `linbo-remote_runs.jsonl` im LINBO-Logverzeichnis
- Exit-Code bei Abbruch durch Signal: 128 + Signal
- `--dry-run` verlangt `-c` oder `-p`

Windows-Treiberprofile (in Entwicklung)

- LINBO ordnet Windows-Treiber **anhand der Hardware-Kennung** jedes Clients automatisch zu und verteilt sie, **ohne zusätzlichen Dienst oder Netzwerk-Port**
- Profile werden über die **API** oder im **Dateisystem** angelegt und Images zugewiesen
- Stand: grundlegend implementiert, Bedienung noch spartanisch, **keine** Oberfläche in WebUI und CLI

> Einmalige Vorbereitung am **Muster-Client** und anschließende Neuerstellung des Images nötig. Ohne sie werden Treiber erst bei der nächsten Anmeldung eines Administrators installiert.

Anleitung: `docs/windows-treiberprofile-einrichten.de.md` im Repository linuxmuster-linbo7

Verbesserungen im Client-Betrieb

- **WLAN in LINBO** zuverlässiger, z. B.:
  - Intel-WLAN: Firmware liegt unter `/lib/firmware/intel/iwlwifi/`, der Kernel sucht sie über einen Link in `/lib/firmware/`. `update-linbofs` legt den Link jetzt mit an (#164).
  - WPA2: Modul `ccm` (AES-Verschlüsselung) fehlte, Verbindung brach mit `key addition failed` ab (#150)
  - Fehlende Firmware wird über die Kernel-Meldung `Direct firmware load for … failed` erkannt (#154), aus der Firmware-Liste entfernte Einträge verschwinden wieder (#153)
- **USB-Netzwerkadapter**, z. B. RTL8153 (#167): Die Netzwerkkarte ers.cheint erst einige Sekunden nach dem Start, LINBO fand noch keine und blieb offline. Jetzt wartet LINBO bis zu 10 Sekunden auf eine Netzwerkschnittstelle, einstellbar mit dem Kernelparameter `netwait=<n>`.
- `init.sh` hängt nicht mehr in `udhcpc` (#187): Auf einer Netzwerkschnittstelle, die nicht aktiv war, wiederholte `udhcpc` endlos `Network is down`, LINBO kam nie über „Requesting ip address per dhcp“ hinaus. Jetzt wird die Schnittstelle übersprungen, `udhcpc` nach spätestens `dhcpretry` × 4 + 5 Sekunden abgebrochen.
- **Cache-Partition** auch ohne `by-id`-Links (#189): Beim Start ohne Server sucht LINBO den Cache über `/dev/disk/by-id`. Fehlen dort Links (virtio-Disk ohne Seriennummer, manche eMMC- und USB-Geräte), fand es nichts und landete im Fernsteuermodus. Jetzt sucht es zusätzlich in allen Partitionen, die der Kernel kennt.

Verbesserungen auf dem LINBO-Server

- **Maschinen-Account-Dateien** nur noch root-lesbar (#182): `*.macct` enthalten die Passwort-Hashes des Maschinenkontos. `linbo-torrent` gab Kopien in `backups/` und `.incoming/` an `nobody`, und das rsync-Modul `[linbo]` liefert als `nobody` ohne Anmeldung aus. Damit waren sie für jeden im Netz abrufbar. Jetzt immer `root:root` mit Rechten `600`.
- `linbo-torrent` und `linbo-multicast` **beenden keine fremden tmux-Sessions** mehr: Ein Neustart der Dienste beendete alle tmux-Sessions des Servers, auch die, in der gerade `linuxmuster-release-upgrade` lief. Jetzt trifft der Neustart nur noch die eigenen Prozesse.
- **Parallele Status- und Log-Uploads** (#172): Alle Clients nutzten dieselbe Zwischendatei in `/srv/linbo/tmp`. Luden zwei gleichzeitig hoch, landeten Daten im Log des falschen Rechners. Jetzt hat jede Übertragung ihre eigene Zwischendatei.
- **GRUB-Images in Multi-Schul-Umgebungen** (#178): `import-devices` trägt Rechner anderer Schulen mit dem Schulnamen als Präfix in DHCP und hostcfg ein, z. B. `gsnord-pc01`. `linbo-mkgrubimg` kannte das nicht, legte das Image unter dem falschen Namen an, und `--setfilename` änderte nichts. Jetzt mit Präfix.

6 · Betrieb und Fehlersuche

Logs und Werkzeuge

Was	Wo
Setup-Module	`/var/log/linuxmuster/setup.<modul>.log`
Geräteimport	`/var/log/linuxmuster/import-devices.log`
Subnetzimport	`/var/log/linuxmuster/import-subnets.log`
Zertifikate	`/var/log/linuxmuster/renew-certs.log`
`linbo-remote`	`<host>_linbo-remote_<ts>.log` + `linbo-remote_runs.jsonl`

Typische Symptome nach dem Upgrade

Symptom	Maßnahme
HTTP 401 von der Firewall	`linuxmuster-opnsense-apikey --check` / `--deploy`
Statische Routen/NAT weg	`linuxmuster-import-subnets`
dhcpd nach Boot nicht aktiv	`systemctl status isc-dhcp-server` : postinst-Fix prüfen
Zertifikatswarnung im Browser	SAN prüfen, `linuxmuster-renew-certs`

7 · Zusammenfassung

Das Wichtigste auf einer Folie

1. **Ubuntu 26.04**-Basis, Umstieg nur gemeinsam mit dem Upgrade (`linuxmuster-release-upgrade`)
2. **OPNsense ≥ 26** und migrierte Regeln als Voraussetzung
3. Firewall im Betrieb per **REST-API**, neues Werkzeug `linuxmuster-opnsense-apikey`
4. **Zertifikate**: SAN, CA-Erneuerung repariert
5. Dateisystem-Quota im **journalled** oder **writeback** Mode
6. LINBO: **Ubuntu-Kernel, aria2c**, Version **7.4.x**, `linbo-remote` in Python mit `--dry-run`
7. **Windows-Treiberprofile** (in Entwicklung)

Ressourcen und Fragen

- Release Notes: `docs/release-notes-linuxmuster.net-74.de.md` (base7)
- Upgrade-Anleitung: `docs/upgrade-26.04.de.md`
- Dokumentation: https://docs.linuxmuster.net
- Forum: https://ask.linuxmuster.net
- Quellcode: https://github.com/linuxmuster

Fragen?

Signed-off by: Thomas Schmitt <thomas@linuxmuster.net>

Assisted by: Claude